

Documentación Técnica del Frontend

Arelys Ajila Douglas Carreño Viviana Cordova
Fabricio Ruiz Jasiel Quezada

2024-06-01

Introducción y Objetivos

El frontend de **MediCampus** es una aplicación web desarrollada con **React 18**, **TypeScript** y **Vite**, diseñada para gestionar el sistema integral de Bienestar Universitario de la Universidad Nacional de Loja.

Objetivos principales:

- Proveer una interfaz moderna, responsiva y accesible para la gestión de citas médicas, historias clínicas, reportes y notificaciones.
- Implementar un sistema de autenticación seguro basado en JWT con control de roles (Administrador, Médico, Paciente).
- Generar reportes dinámicos con gráficos estadísticos (Chart.js) y exportación a PDF/CSV.
- Garantizar la trazabilidad de operaciones mediante auditoría integrada.

Arquitectura del Proyecto

El proyecto sigue una arquitectura basada en módulos funcionales, cada uno con su propia estructura de carpetas:

```
src/  
  main.tsx           # Punto de entrada  
  App.tsx            # Configuración de rutas  
  api.ts             # Cliente HTTP raíz (fetch)  
  ui/  
    components/  
      Button.tsx  
      Card.tsx  
      Modal.tsx
```

```

    Input.tsx
    Select.tsx
    Table.tsx
    Badge.tsx
    Pagination.tsx
    Sidebar.tsx
    Topbar.tsx
    SearchInput.tsx
    ToggleSwitch.tsx
global/                                # Páginas públicas
    LandingPage.tsx
    HomePage.tsx
    AccessDenied.tsx
seguridad/                             # Módulo de autenticación y seguridad
    context/
    hooks/
    utils/
    components/
    views/
reportes/                              # Módulo de reportes y gráficos
    hooks/
    service/
    component/
    print/
    utils/
historias-clinicas/                   # Módulo de historias clínicas
    hooks/
    services/
    components/
    pages/
    utils/
agendas/                             # Módulo de agendas y citas
    context/
    hooks/
    services/
    components/
    utils/
notificaciones/                       # Módulo de notificaciones
    component/
    test/
theme/
    global.css                        # Estilos base Tailwind

```

Lógica de interacción entre capas:

1. **Páginas (pages/ o views/)** → orquestan hooks y componentes.
2. **Hooks personalizados (hooks/)** → manejan estado local y llamadas a servicios.
3. **Servicios (service/ o services/)** → encapsulan llamadas HTTP (Axios/fetch).
4. **Componentes de UI (component/)** → presentación pura, reciben props.
5. **Contextos (context/)** → estado global compartido (autenticación).

Flujo de Trabajo

Gestión de Estado

El proyecto utiliza **React Context** para estado global y **hooks personalizados** para estado local:

- **AuthContext** (seguridad/context/AuthContext.tsx): Provee `token`, `user`, `isAuthenticated`, `isAdmin`, `login()` y `logout()` a toda la aplicación.
- **AgendaContext** (agendas/context/AgendaContext.ts): Estado específico del módulo de agendas.
- **Hooks de datos**: Cada módulo define hooks que encapsulan la lógica de fetching (`useReportesDashboard`, `useHistoriasClinicas`, `useAgenda`, etc.).

Ciclo de vida de una operación típica:

1. El usuario interactúa con un componente de UI (ej. hace clic en “Exportar PDF”).
2. El componente llama a una función del hook (ej. `handleExport('pdf')`).
3. El hook actualiza estado local (`loading=true`) y llama al servicio API.
4. El servicio realiza la petición HTTP (Axios/fetch) con el token JWT.
5. Al recibir la respuesta, el hook actualiza el estado y el componente se re-renderiza.

Protección de rutas:

- **AdminGuard**: Verifica autenticación y rol admin antes de acceder a `/reportes`.
- **AuthGuard**: Verifica token JWT válido para rutas de agendas.
- **ProfessionalGuard**: Verifica rol profesional para rutas de consulta médica.

Guía de Estilos

Integración con Tailwind CSS

El proyecto utiliza **Tailwind CSS v3** con configuración personalizada:

```
// tailwind.config.js
export default {
  content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],
  theme: {
    extend: {
      colors: {
        hc: {
          bg: '#f9f9ff',
          primary: '#003f87',
          primaryText: '#ffffff',
          secondary: '#e9eefc',
          danger: '#ba1a1a',
          success: '#006a61',
        },
      },
      borderRadius: { global: '0.75rem' },
      fontFamily: { sans: ['Inter', 'ui-sans-serif', 'system-ui', 'sans-serif'] },
    },
  },
  plugins: [],
};
```

Componentes base

Los componentes base en `src/ui/components/` siguen estos principios:

- **Composables:** Table se compone de TableHeader, TableBody, TableRow, TableHead, TableCell.
- **Variantes:** Button soporta 5 variantes (primary, secondary, tertiary, danger, success) y 3 tamaños (sm, md, lg).
- **Accesibilidad:** ToggleSwitch usa aria-pressed; Modal maneja foco y cierre con Escape.
- **Consistencia:** Todos los componentes usan las variables de color hc-* definidas en Tailwind.

Esquema de API (Conexión)

El frontend consume el backend Django a través de los siguientes clientes HTTP:

Configuración de proxy (Vite)

```
// vite.config.ts
proxy: {
  '/backend': {
    target: 'http://127.0.0.1:8000',
    changeOrigin: true,
    rewrite: (p) => p.replace(/^\/backend/, ''),
  },
  '/api': {
    target: 'http://127.0.0.1:8000',
    changeOrigin: true,
  },
}
```

Módulos y endpoints

Módulo	Cliente	Base URL	Método
Seguridad	fetch (authApi)	/api/v1/auth	login, register, getMe, CRUD usuarios/roles
Reportes	Axios (reportService)	/backend/api/v1/reports	estadísticas, atenciones, consultas-por-genero, export
Agendas	Axios (axiosConfig)	VITE_API_BASE_URL o /api	CRUD citas, consultas, derivaciones
Historias Clínicas	fetch (SimpleApiClient)	VITE_API_BASE_URL o /api/v1	CRUD historias, antecedentes, documentos
Notificaciones	fetch	depende de API_CONFIG	notificaciones

Autenticación

Cada petición incluye el token JWT en el header `Authorization: Bearer <token>`. El token se almacena en `localStorage` bajo las claves `token`, `refreshToken` y `currentUser`.

Los interceptores de Axios manejan automáticamente: - Agregar el token a cada petición (request interceptor). - Redirigir al login en caso de 401 (response interceptor).

Formato de respuesta

Todas las APIs utilizan un wrapper uniforme:

```
interface ApiWrapper<T = any> {  
  success: boolean;  
  message: string;  
  data: T | null;  
  errors?: Record<string, any> | null;  
}
```

Conclusión

El frontend de MediCampus está construido con una arquitectura modular y escalable que separa claramente las responsabilidades en páginas, hooks, servicios y componentes. El uso de TypeScript proporciona tipado estático que reduce errores en tiempo de ejecución, mientras que Tailwind CSS agiliza el desarrollo de interfaces consistentes.

Próximos pasos sugeridos:

1. Unificar los distintos clientes HTTP bajo una sola instancia Axios centralizada.
2. Consolidar el almacenamiento de tokens de autenticación bajo un solo par de claves.
3. Completar la implementación de exportación PDF del lado del servidor.
4. Migrar los tests a Vitest con cobertura mínima del 80 %.
5. Implementar pruebas E2E con Playwright o Cypress para los flujos críticos.